

Informe Técnico de Arquitectura y Componentes

Proyecto: Fate-Stay-Night-main

Tipo de Documento: Memoria de Desarrollo y Automatización

Ecosistema: Java (Maven) / GitHub Actions Automation

1. Resumen del Proyecto

El repositorio analizado corresponde a una estructura de proyecto Java administrada mediante Maven que integra un sistema automatizado de modernización y actualización continua a través de scripts de CI/CD e integración de hooks. El proyecto combina la lógica de construcción estandarizada con herramientas automatizadas de refactorización de código Java.

2. Desglose Estructural del Repositorio

Ruta del Archivo / Directorio	Tipo	Propósito y Función Técnica
pom.xml	Configuración	Archivo principal de configuración de Maven (POM). Define dependencias, plugins de compilación y metadata del proyecto.
README.md	Documentación	Guía general del proyecto, instrucciones de uso, configuración del entorno y requisitos previos.
docs/DIAGRAMAS.puml	PlantUML	Diagramas de arquitectura, componentes y secuencia modelados en sintaxis PlantUML para documentación visual.
docs/MEMORIA_TECNICA.md	Documentación	Detalle de especificaciones técnicas, decisiones de diseño, arquitectura y manuales de mantenimiento.
.github/modernize/java-upgrade/	Automatización	Entorno automatizado para la migración y modernización del código fuente a versiones recientes de Java.
hooks/scripts/recordToolUse.ps1	PowerShell	Script para capturar y registrar métricas de uso de herramientas durante el flujo de trabajo en Windows.
hooks/scripts/recordToolUse.sh	Bash	Script para captura y auditoría de uso de herramientas automatizadas en entornos Linux/Unix.
rin tohsaka.mp4	Multimedia	Recurso de video complementario almacenado en la raíz del proyecto.

3. Detalle de los Módulos de Automatización (Modernize)

El flujo ubicado en `.github/modernize/java-upgrade/` está diseñado para automatizar las tareas repetitivas de actualización de sintaxis, dependencias y refactorización de patrones obsoletos en Java.

Hooks & Registros (`recordToolUse`)

Garantizan la trazabilidad del proceso de migración. Permiten registrar cuándo y cómo se ejecutan las herramientas de análisis estático o refactorización automática en el pipeline.

Documentación y Diagramado

Uso de PlantUML (`DIAGRAMAS.puml`) y Markdown para mantener la coherencia entre la implementación de código y los diagramas estructurales del sistema.

4. Flujo de Trabajo e Integración

El proyecto sigue un estándar de desarrollo con entrega continua:

1. Definición de Construcción (`pom.xml`) 2. Ejecución de Hooks de Modernización (`.github/modernize`) 3. Auditoría e Historial de Ejecución (`recordToolUse.ps1 / .sh`) 4. Verificación de Memoria Técnica y Diagramas UML

5. Conclusión

El proyecto integra una estructura organizada que combina la construcción mediante Maven con capacidades de modernización continua y registro automatizado. La presencia de documentación técnica detallada asegura la mantenibilidad del código a lo largo de sus distintas fases de actualización.

